

The Master Architecture Blueprint

Complete End-to-End System Architecture, Microservice Dataflows & Code Implementations

System Type:	3-Tier AI-Powered Adaptive Education Engine	Target LLM:	Google Gemini 2.5 / 1.5 Flash (GenAI SDK)
Frontend:	React 18 + Vite + Tailwind CSS	Backend:	Spring Boot 3 (Java 21) + Hibernate + MySQL/H2
AI Microservice:	FastAPI (Python 3.11) + Pydantic v2	ML Telemetry:	Scikit-Learn (RandomForestClassifier)

1. Executive Architectural Summary

AdaptiveLearn is an enterprise-grade AI education platform engineered to transform arbitrary career objectives (e.g., *'Java Backend Developer with Spring Boot'*, *'AWS Cloud DevOps'*, or *'IELTS Preparation'*) into structured, milestone-based learning roadmaps. The system features real-time **Socratic mentoring**, **automated remediation loops** that dynamically heal knowledge gaps when assessment scores drop below 70%, and **machine learning telemetry** that predicts student dropout risks in real-time.

TERMINAL / CODE VIEW: TOPOLOGY DIAGRAM



Why Separate Spring Boot and FastAPI? (Design Justification)

Domain / Concern	Technology Choice	Technical Justification
Security & Identity	Spring Boot (Java)	Industry standard for bulletproof JWT security filters, BCrypt password hashing, and role-based route guards.
Data Integrity & ACID	Spring Data JPA / MySQL	Manages complex relational entities (@OneToMany, @ManyToOne), cascade operations, and transactional database integrity.
AI & LLM Orchestration	FastAPI (Python)	Native runtime for Google GenAI SDK, Pydantic strict schemas, numpy, and asynchronous generative model streaming.
Machine Learning	Scikit-Learn (Python)	Trains and runs real-time RandomForestClassifier pipeline on student behavioral telemetry.

2. Feature 1: User Authentication & JWT Security Workflow

■ **REAL-WORLD ANALOGY:** Think of an airport security passport check. You show your passport once at the check-in desk (Registration/Login). The airline issues you a digitally signed boarding pass (JWT Token). For every gate you board afterwards (accessing courses or AI tutor), you show your boarding pass. Security verifies the signature in milliseconds without re-querying the passport database.

Step 1 [React UI (Login.jsx)]: User inputs email/password -> Axios dispatches HTTP POST /api/auth/login to Spring Boot.

Step 2 [Spring Boot (AuthController)]: Authenticates credentials with AuthenticationManager -> Verifies BCrypt hashed password against database.

Step 3 [Spring Boot (JwtUtils)]: Generates cryptographically signed JWT token embedding student email and ROLE_STUDENT claims.

Step 4 [React (AuthContext.jsx)]: Saves JWT in localStorage -> Configures Axios Request Interceptor to inject 'Authorization: Bearer ' on all future requests.

Step 5 [Spring Boot (Security Filter)]: JwtAuthenticationFilter intercepts incoming requests -> Validates signature -> Sets SecurityContextHolder authentication.

TERMINAL / CODE VIEW: AUTHENTICATION & TOKEN INTERCEPTOR

```
// 1. Spring Boot Controller: Authenticating and Generating JWT
@PostMapping("/login")
public ResponseEntity login(@RequestBody AuthDTO.LoginRequest request) {
    Authentication auth = authManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.getEmail(), request.getPassword())
    );
    String token = jwtUtils.generateToken(request.getEmail());
    User user = userRepository.findByEmail(request.getEmail()).orElseThrow();
    return ResponseEntity.ok(new AuthDTO.AuthResponse(token, user.getId(), user.getEmail(), user.getFullName()));
}

// 2. React Axios Interceptor: Automatically attaching Token
api.interceptors.request.use((config) => {
    const token = localStorage.getItem('token');
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
});
```

3. Feature 2: Onboarding & Baseline Diagnostic Pre-Assessment

■ **REAL-WORLD ANALOGY:** Think of a personal fitness trainer. Before creating a customized workout routine, the trainer tests your baseline strength on 3 exercises to determine what you already know. This avoids wasting time teaching you beginner basics if you already possess intermediate competencies.

Step 1 [React (OnboardingWizard)]: Collects student goal, stated experience level, available hours per week, and preferred learning style.

Step 2 [Spring Boot (/student/onboarding)]: Persists preferences -> Invokes FastAPI /ai/generate-diagnostic-assessment via AiServiceClient.

Step 3 [FastAPI & Gemini LLM]: Fills DIAGNOSTIC_ASSESSMENT_PROMPT -> Generates 3-5 multi-topic MCQs calibrated to the student's target goal.

Step 4 [React (DiagnosticQuiz.jsx)]: Presents interactive testing environment -> Evaluates individual topic scores (e.g. OOP: 100%, SQL: 33%).

Step 5 [Course Generator (/courses/generate)]: Transmits topic scores to synthesis engine -> Builds curriculum that skips mastered concepts.

TERMINAL / CODE VIEW: FASTAPI DIAGNOSTIC ENDPOINT

```
# FastAPI: Generating Structured Diagnostic Questions via Gemini
@app.post("/ai/generate-diagnostic-assessment", response_model=DiagnosticAssessmentResponse)
async def generate_diagnostic_assessment(req: DiagnosticAssessmentRequest):
    prompt = DIAGNOSTIC_ASSESSMENT_PROMPT.format(
        goal=req.goal,
        experience_level=req.experience_level
    )
    # Enforces strict Pydantic schema validation on LLM JSON output
    return await llm_adapter.generate_structured(prompt, DiagnosticAssessmentResponse)
```

4. Feature 3: Course Roadmap Synthesis, AI Re-Tuning & Goal Pivot

■ **REAL-WORLD ANALOGY:** Think of a GPS navigation system. It calculates the fastest multi-stop route to your destination. If you hit a roadblock or decide midway to change your destination from 'Chicago' to 'New York', the GPS preserves all the miles you've already driven and recalculates only the road ahead.

Step 1 [Student Goal Input]: Student enters objective: 'Java Backend Developer with Spring Boot & Microservices'.

Step 2 [Spring Boot -> FastAPI Call]: LearningPathService calls FastAPI /ai/create-course-roadmap via Spring RestClient HTTP POST.

Step 3 [FastAPI -> Gemini Generation]: Gemini decomposes the objective into 4-6 sequential milestone modules with estimated durations and difficulty.

Step 4 [Database Persistence (JPA)]: Spring Boot saves LearningPath entity and cascades LearningPathItem entities (Module 1 UNLOCKED, others LOCKED).

Step 5 [Mid-Track Goal Pivot (/adapt-goal)]: If student changes goal, completed modules are preserved; uncompleted modules are dynamically regenerated by AI.

TERMINAL / CODE VIEW: SPRING BOOT ROADMAP SYNTHESIS

```
// Spring Boot: LearningPathService.java - Persisting Course & Modules
@Transactional
public LearningPath createCourseRoadmap(String email, Map request) {
    StudentProfile profile = profileRepository.findByUserEmail(email).orElseThrow();
    Map aiResponse = aiServiceClient.createCourseRoadmap(request);
    List<String> modules = (List<String>) aiResponse.get("modules");

    LearningPath path = pathRepository.save(new LearningPath(profile, (String) aiResponse.get("course_title"),
    modules.size()));
    int seq = 1;
    for (Map m : modules) {
        LearningPathItem.ItemStatus status = (seq == 1) ? ItemStatus.UNLOCKED : ItemStatus.LOCKED;
        pathItemRepository.save(new LearningPathItem(path, (String)m.get("title"), (String)m.get("topic_name"),
        (String)m.get("description"), seq, DifficultyLevel.BEGIN
        NER, 60, status));
        seq++;
    }
    return path;
}
```

5. Feature 4: Deep Interactive Lesson Notes & Video Curations

■ **REAL-WORLD ANALOGY:** Think of an intelligent digital textbook that writes itself on-demand. When you open a chapter for the first time, an expert professor writes comprehensive theory, provides working code examples, warns you of common mistakes, and attaches curated video lectures. The chapter is then cached in the library for instant 5ms loading on future visits.

TERMINAL / CODE VIEW: SMART LESSON CACHING PIPELINE

```
// Spring Boot: Smart Database Caching for Lesson Content
@Transactional
public Map getOrGenerateLessonContent(Long itemId, String email) {
    LearningPathItem item = pathItemRepository.findById(itemId).orElseThrow();

    // 1. Return from DB cache if previously generated (5ms response time)
    if (item.getLessonContentJson() != null && !item.getLessonContentJson().isBlank()) {
        return objectMapper.readValue(item.getLessonContentJson(), Map.class);
    }

    // 2. Cache miss: Request FastAPI /ai/generate-lesson-content
    Map contentRes = aiServiceClient.generateLessonContent(Map.of(
        "topic_name", item.getTopicName(), "course_goal", item.getLearningPath().getGoalText()
    ));

    // 3. Cache into Database and return
    item.setLessonContentJson(objectMapper.writeValueAsString(contentRes));
    pathItemRepository.save(item);
    return contentRes;
}
```

6. Feature 5: Real-Time Socratic AI Tutor & Workspace Sandbox

■ **REAL-WORLD ANALOGY:** Standard AI chatbots give you direct answers immediately, encouraging passive reading. A Socratic mentor acts like an Oxford tutor: it responds with thought-provoking questions, real-world mechanical analogies, and targeted hints that guide you to discover the fundamental architectural principles yourself.

TERMINAL / CODE VIEW: FASTAPI SOCRATIC MENTORING ENGINE

```
# FastAPI: main.py - Context-Aware Socratic AI Tutor Prompt
@app.post("/ai/chat", response_model=ChatResponse)
async def chat_with_tutor(req: ChatRequest):
    system_instruction = f"""
    You are an expert Socratic AI Tutor for the topic: '{req.topic_name}'.
    Student Level: {req.current_level} | Known Weaknesses: {req.weak_skills}
    Pedagogy Rules:
    1. Guide the student using relatable real-world physical analogies and mental models.
    2. Never give raw unguided answers; prompt interactive active-recall questions.
    3. When code is requested, provide clean syntax-highlighted examples with explanations.
    """
    reply = await llm_adapter.generate_chat_text(system_instruction, req.message, req.conversation_history)
    return ChatResponse(reply=reply, suggested_followups=[
        f"Explain {req.topic_name} with another analogy", "Give me a 1-minute conceptual challenge!"
    ])
```

7. Feature 6: Multi-Modal Assessment & Automated Remediation Loop (< 70%)

■ **REAL-WORLD ANALOGY:** In traditional learning, when a student fails a test, they are told to 'try again' with no assistance. In AdaptiveLearn, if your score drops below 70%, the system activates a self-healing loop: AI analyzes your exact mistakes, creates a custom 'Remedial Mastery Lab' module, and automatically inserts it into your active roadmap to reinforce prerequisites.

Step 1 [Student Submits Test]: Submits MCQs, Conceptual Descriptive Essay, and Sandbox Code to POST /api/assessments/{id}/submit.

Step 2 [Rubric Grading Engine]: Spring Boot grades MCQs & invokes FastAPI /ai/evaluate-descriptive for LLM rubric evaluation.

Step 3 [Branch A: Score >= 70%]: Module marked COMPLETED -> Next sequential module unlocked -> Skill Mastery Radar updated -> Confetti 🎉.

Step 4 [Branch B: Score < 70%]: Spring Boot triggers Remediation Loop -> Calls FastAPI /ai/generate-remedial-module with specific weaknesses.

Step 5 [Dynamic Roadmap Insertion]: Spring Boot shifts all subsequent modules by +1 in sequence order -> Inserts Remedial Lab -> Unlocks it immediately.

TERMINAL / CODE VIEW: AUTOMATED REMEDIATION LOOP

```
// Spring Boot: AssessmentService.java - Automated Remediation Trigger Logic
if (percentage >= 70.0) {
    learningPathService.completeItemAndUnlockNext(assessment.getPathItem().getId(), email);
} else {
    // AUTOMATED REMEDIATION SELF-HEALING LOOP
    Map remedialData = aiServiceClient.generateRemedialModule(Map.of(
        "topic_name", assessment.getPathItem().getTopicName(),
        "score_percentage", percentage,
        "weaknesses", extractedWeaknesses,
        "course_goal", assessment.getPathItem().getLearningPath().getGoalText()
    ));
    // Dynamically inserts module into student's roadmap tree
    learningPathService.insertRemedialItem(path.getId(), assessment.getPathItem(), remedialData);
}
```

8. Feature 7: Scikit-Learn Predictive Dropout Risk & Skill Matrix

■ **REAL-WORLD ANALOGY:** Think of an airplane cockpit flight recorder (black box). It continuously analyzes speed, altitude, weather turbulence, and pitch. If it detects conditions that lead to stalls, it issues early warnings to the pilot. Our Scikit-Learn ML classifier monitors study velocity, missed quizzes, and inactivity to intervene before a student gives up.

Step 1 [Completion Pace Rate]: Proportion of total course milestones completed versus expected timeline (0.0 to 1.0).

Step 2 [Assessment Average]: Cumulative score average across all objective, descriptive, and sandbox assessments.

Step 3 [Inactivity Duration]: Number of days elapsed since the student's last active learning session.

Step 4 [Missed Checkpoints]: Number of skipped or failed milestone verification tests.

Step 5 [Performance Trend Slope]: Mathematical gradient slope of recent quiz scores (positive growth vs. negative decline).

TERMINAL / CODE VIEW: SCIKIT-LEARN RISK CLASSIFIER

```
# FastAPI: main.py - Scikit-Learn RandomForest Predictive Pipeline
class LearningRiskPredictor:
    def __init__(self):
        self.pipeline = Pipeline([
            ('scaler', StandardScaler()),
            ('rf', RandomForestClassifier(n_estimators=50, random_state=42))
        ])
        self.pipeline.fit(X_train, y_train)

    def predict(self, completion_rate, avg_score, days_inactive, missed_count, trend_slope):
        features = np.array([[completion_rate, avg_score, days_inactive, missed_count, trend_slope]])
        pred_class = self.pipeline.predict(features)[0] # 0=LOW, 1=MEDIUM, 2=HIGH
        proba = self.pipeline.predict_proba(features)[0]
        risk_score = float(proba[1]*0.5 + proba[2]*1.0)
        risk_level = {0: 'LOW', 1: 'MEDIUM', 2: 'HIGH'}.get(pred_class, 'LOW')
        return risk_level, risk_score, self._calculate_factors(features)
```

9. Architectural Summary Matrix

Feature Area	Frontend UI (React)	Backend Logic (Spring Boot)	AI & ML Microservice (FastAPI)
1. Authentication	JWT in localStorage & Axios Interceptor	Spring Security + BCrypt + JwtUtils	N/A (Decoupled Auth)
2. Baseline Quiz	DiagnosticQuiz.jsx with Topic Meters	Persists Onboarding Preferences	Gemini Dynamic MCQ Generator
3. Roadmap Tree	CourseRoadmap.jsx (Progression Tree)	LearningPath & Items State Machine	Gemini Curriculum Decomposer
4. Lesson Notes	LessonViewer.jsx + Dark Code Box	Smart DB JSON Caching (5ms load)	Markdown + YouTube Curator
5. Socratic Tutor	ChatWorkspace.jsx + Code Sandbox	Conversation Session Orchestrator	Context-Aware Analogy Prompt Engine
6. Remediation	AssessmentRunner.jsx + Remedial Alert	Auto-inserts Remedial Module (<70%)	Synthesizes Targeted Practice Lab
7. ML Telemetry	StudentProfile.jsx (Proficiency Radar)	Aggregates Velocity & Test Metrics	Scikit-Learn RandomForest Pipeline